Speed Control of a DC Motor Using PI Controller

Submitted by:

Mohammad Maher Ibrahim	202301837
Amira Sherif	202300076
Ahmad Essam	202301992

University of Science and Technology in Zewail City

CIE 318: Control Systems

Under the supervision of:

Dr. Abdel-Latif Elshafei

Fall 2025

Contents

Abstract **3**

1 Introduction **3**

2 Problem Description and Solution Techniques **3**

2.1 Problem Description . . . 3

2.2 Solution Techniques . . . 4

2.2.1 Open-Loop Control . . . 4

2.2.2 Proportional (P) Control . . . 4

2.2.3 Proportional–Integral (PI) Control . . . 4

2.2.4 Proportional–Integral–Derivative (PID) Control . . . 5

2.2.5 Advanced Control Techniques . . . 5

2.3 Selected Approach . . . 5

3 Hardware and Simulink Setup **5**

3.1 Hardware Components . . . 5

3.1.1 Arduino Uno . . . 5

3.1.2 H-Bridge Motor Driver (L298N) . . . 6

3.1.3 DC Motor with Encoder . . . 7

3.2 Simulink Model Design . . . 8

3.2.1 Open-Loop System Characterization Model . . . 8

3.2.2 Closed-Loop PI Control Model . . . 9

4 Motor Direction Verification **9**

5 Speed Measurement and Filtering **10**

6 Effect of Filter Time Constant **10**

6.1 First-Order Low-Pass Filter Design . . . 11

6.2 Case 1: $\tau = 0.2$ s . . . 11

6.3 Case 2: $\tau = 0.5$ s . . . 11

6.4 Case 3: $\tau = 1$ s . . . 11

6.5 Case 4: $\tau = 2$ s . . . 11

6.6 Forward Direction Response . . . 12

6.7 Reverse Direction Response . . . 13

6.8 Verification Using MATLAB . . . 13

7 System Identification **14**

7.1 Tested Transfer Function Structures . . . 14

7.1.1 Case 1: Second-Order System with One Zero . . . 14

7.1.2 Case 2: Second-Order System without Zeros 14
7.1.3 Case 3: Third-Order System with Two Zeros 14
7.1.4 Case 4: First-Order System 15
7.2 Pulse Response Comparison 15
7.3 Model Selection 16

8 PI Controller Design **16**
8.1 System Model and Controller Structure 16
8.2 Closed-Loop Characteristic Equation 16
8.3 Design Based on Settling Time Specification 17
8.4 Controller Design for Different Overshoot Specifications 18
8.4.1 Case 1: Maximum Overshoot = 5% 18
8.4.2 Case 2: Maximum Overshoot = 10% 19
8.5 Experimental Validation 19

9 Effect of Different Speed Setpoints **20**
9.1 Setpoint Tracking Performance 20
9.2 Analysis and Discussion 21

10 Conclusion **22**

References **23**

Abstract

This project presents the design and implementation of a proportional–integral (PI) controller for the speed control of a low-power DC motor using an Arduino microcontroller and an H-bridge driver. The motor speed is measured using an encoder and processed through a low-pass filter to reduce noise. System identification techniques are applied to obtain a mathematical model of the motor. Based on the identified model, a PI controller is designed to achieve a closed-loop settling time equal to 80% of the open-loop settling time. The controller is first designed in continuous time and then converted to discrete time using backward transformation. Experimental and simulation results are presented and analyzed to evaluate the controller performance.

1 Introduction

DC motor speed control is a fundamental problem in control systems and mechatronics due to its wide range of industrial applications. The objective of this project is to control the speed of a DC motor using a PI controller implemented through MATLAB/Simulink and Arduino hardware.

Several control strategies can be used for motor speed regulation, such as proportional (P), proportional–integral (PI), and proportional–integral–derivative (PID) controllers. Among these, the PI controller offers a good balance between simplicity and performance, providing zero steady-state error while maintaining acceptable transient behavior.

This report describes the hardware setup, Simulink model, system identification process, PI controller design, and performance analysis. The report is organized as follows: Section 2 discusses the problem description and solution techniques; Section 3 presents the hardware and Simulink setup; Section 4 covers motor direction verification; Section 5 addresses speed measurement and filtering; Section 6 analyzes the effect of filter time constants; Section 7 details system identification; Section 8 presents the PI controller design; Section 9 evaluates different speed setpoints; and Section 10 provides the conclusion.

2 Problem Description and Solution Techniques

2.1 Problem Description

The primary objective of this project is to achieve accurate and stable speed control of a DC motor. When operated in open-loop mode, the motor speed varies significantly due to changes in load conditions, supply voltage fluctuations, and inherent system nonlinearities. Moreover, sensor noise from the encoder introduces disturbances in the measured speed signal, which can degrade control performance if not properly handled.

Without feedback control, the system is unable to maintain a desired reference speed or respond effectively to disturbances. Therefore, a closed-loop control strategy is required to ensure reliable speed regulation, fast transient response, minimal steady-state error, and robustness against external disturbances.

The main challenges addressed in this project include:

- Regulating the motor speed to follow a desired reference value
- Reducing the effect of measurement noise using signal filtering
- Achieving a faster closed-loop settling time compared to open-loop operation
- Ensuring stable operation with minimal overshoot
- Implementing the controller in real time using Arduino and Simulink

2.2 Solution Techniques

Several control techniques can be applied to solve the DC motor speed control problem. The most common approaches include:

2.2.1 Open-Loop Control

In open-loop control, the motor speed is controlled by directly adjusting the input voltage or PWM signal without using feedback. Although this approach is simple to implement, it is highly sensitive to disturbances and parameter variations. As a result, open-loop control is not suitable for applications requiring precise speed regulation.

2.2.2 Proportional (P) Control

A proportional controller adjusts the control input based on the instantaneous error between the reference speed and the measured speed. While P control improves response speed, it cannot eliminate steady-state error and may lead to oscillations if the gain is increased excessively.

2.2.3 Proportional–Integral (PI) Control

The PI controller combines proportional and integral actions to improve system performance. The proportional term enhances transient response, while the integral term eliminates steady-state error. PI control offers a good balance between simplicity and effectiveness, making it one of the most widely used controllers in industrial motor control applications. For this reason, PI control was selected for this project.

2.2.4 Proportional–Integral–Derivative (PID) Control

PID control introduces an additional derivative term to improve damping and reduce overshoot. Although PID controllers can provide superior performance, they are more sensitive to noise and require careful tuning. Given the encoder noise and implementation constraints, PID control was not adopted in this project.

2.2.5 Advanced Control Techniques

Advanced methods such as state-space control, adaptive control, and robust control can offer improved performance under varying operating conditions. However, these techniques require more complex modeling, higher computational resources, and increased implementation effort, which are beyond the scope of this project.

2.3 Selected Approach

Based on performance requirements, implementation simplicity, and robustness, a PI controller was selected as the most suitable solution. The controller parameters were designed using control theory principles based on the identified motor model. The controller was first designed in continuous time and then converted to discrete time for real-time implementation using Arduino and Simulink.

3 Hardware and Simulink Setup

The hardware components used in this project include an Arduino Uno, a DC motor with an encoder, an L298N H-bridge driver, and an external DC power supply. The hardware connections and Simulink configuration are described in the following subsections.

3.1 Hardware Components

The experimental setup of the DC motor speed control system consists of the following main hardware components. Each component plays a critical role in ensuring proper control, power delivery, and feedback acquisition.

3.1.1 Arduino Uno

The Arduino Uno serves as the main control unit of the system. It is responsible for executing the control algorithm generated from the Simulink model, generating the PWM control signal, and interfacing with the motor driver. Additionally, the Arduino processes the feedback signals received from the motor encoder.

In this project, digital pins 2 and 3 of the Arduino Uno were used to receive the encoder signals from the DC motor. These pins support external interrupts, which allow accurate measurement of the motor speed and ensure reliable real-time feedback for the control system.

Figure 1: Arduino Uno used as the main controller

3.1.2 H-Bridge Motor Driver (L298N)

The H-bridge driver is used to interface the low-power control signals from the Arduino with the higher power requirements of the DC motor. Since the Arduino cannot supply sufficient current to drive the motor directly, the H-bridge provides the necessary power amplification.

Moreover, the H-bridge enables bidirectional motor operation by controlling the polarity of the voltage applied to the motor terminals. The control inputs of the H-bridge are connected to the Arduino digital output pins, allowing the motor to rotate in both forward and reverse directions as required.

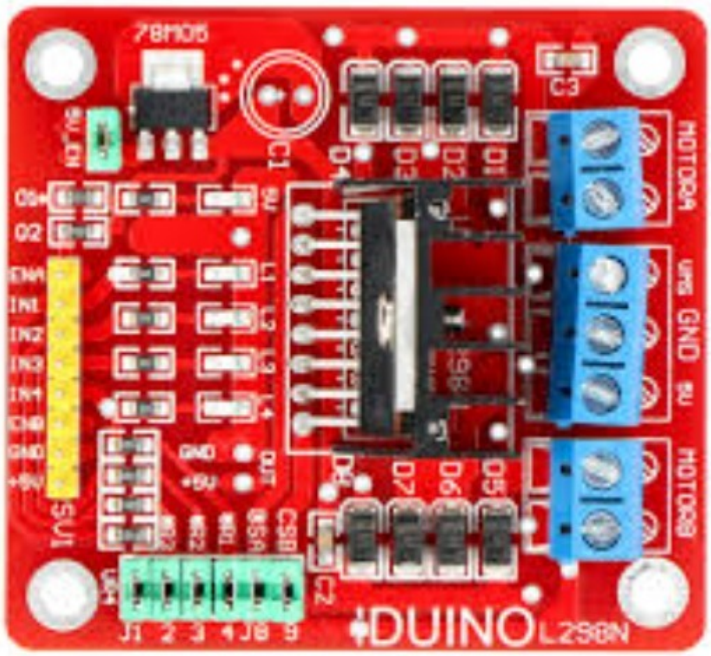


Figure 2: L298N H-bridge motor driver used to power and control the DC motor

3.1.3 DC Motor with Encoder

The DC motor is the plant under control in this project. Its speed is regulated by adjusting the PWM signal applied through the H-bridge. An incremental encoder is attached to the motor shaft to provide real-time feedback of the motor speed.

The encoder outputs are connected directly to pins 2 and 3 of the Arduino Uno. These signals are used to calculate the motor speed in revolutions per minute (RPM), which is then fed back into the control loop to ensure accurate speed regulation.

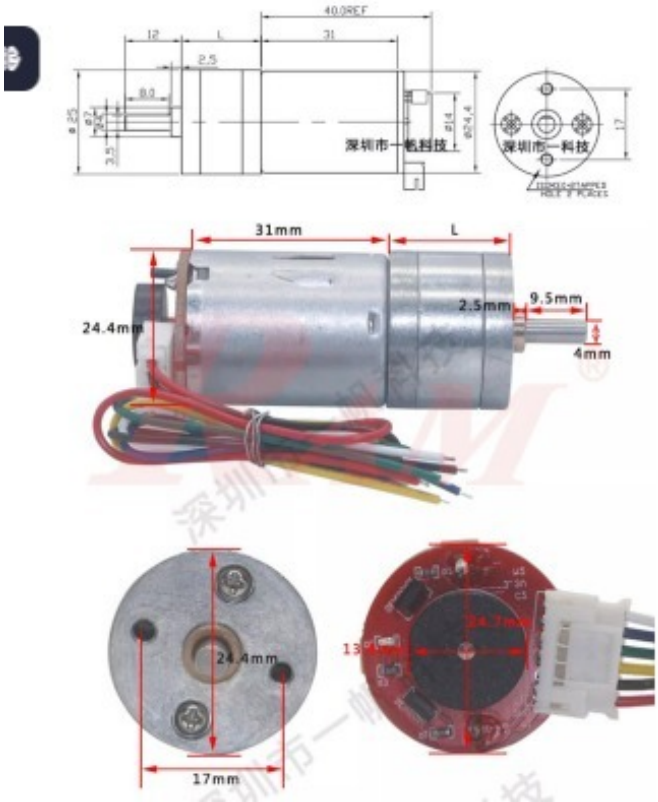


Figure 3: DC motor with attached encoder used for speed feedback

3.2 Simulink Model Design

The control system was implemented using MATLAB/Simulink with Arduino support packages. Two main Simulink configurations were developed: one for initial system characterization and another for closed-loop PI control.

3.2.1 Open-Loop System Characterization Model

The first Simulink model was designed to characterize the open-loop behavior of the DC motor system and to identify its transfer function. This model includes the Arduino interface blocks, encoder reading blocks, PWM generation, and data logging capabilities. The measured speed data from this configuration was used for system identification.

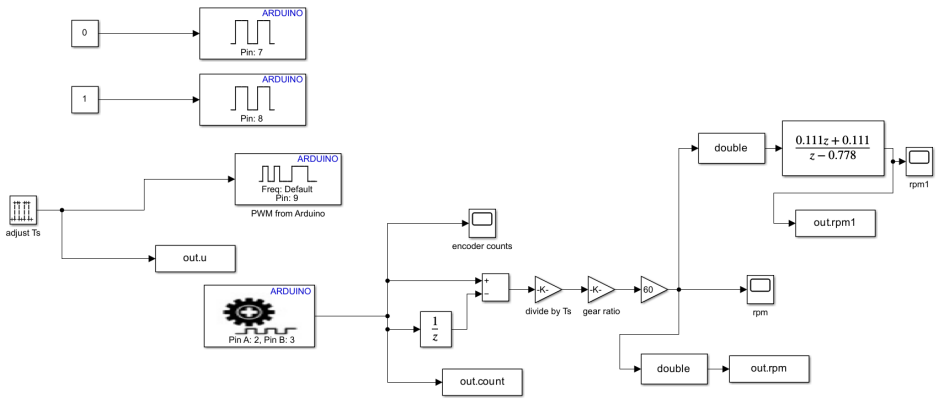


Figure 4: Simulink model for open-loop system characterization and identification

3.2.2 Closed-Loop PI Control Model

After identifying the system transfer function, a closed-loop control model was developed incorporating the designed PI controller. This model includes the reference speed input, error calculation, discrete PI controller block, PWM output, and feedback from the encoder. The controller parameters obtained from the design procedure were implemented in this model.

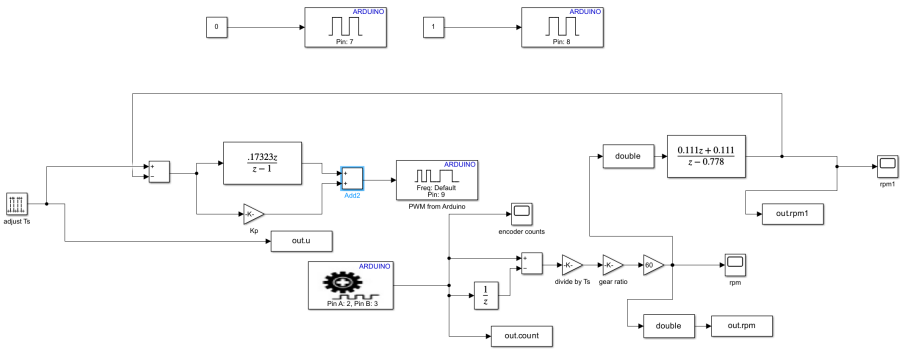
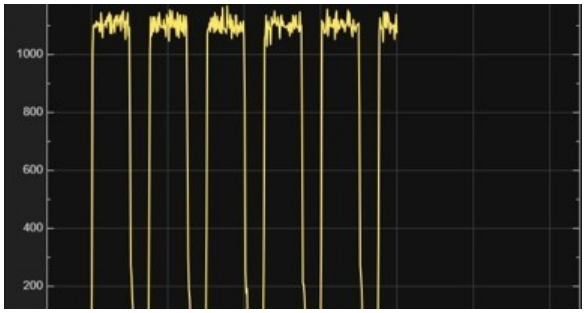


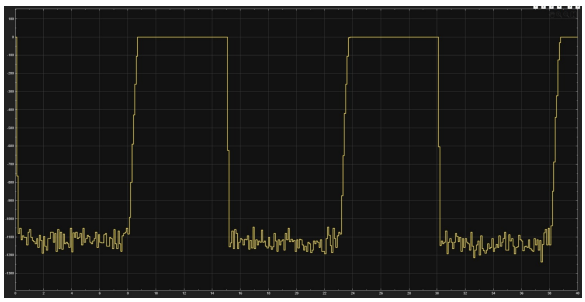
Figure 5: Simulink model for closed-loop PI control implementation

4 Motor Direction Verification

To verify the ability of the motor to rotate in both forward and reverse directions, the control pins IN1 and IN2 were interchanged between digital pins 7 and 8 of the Arduino. By changing the logic levels applied to these pins, the polarity of the voltage across the motor terminals is reversed, resulting in a change in rotation direction. Successful bidirectional operation was confirmed through experimental testing.



(a) Forward direction configuration



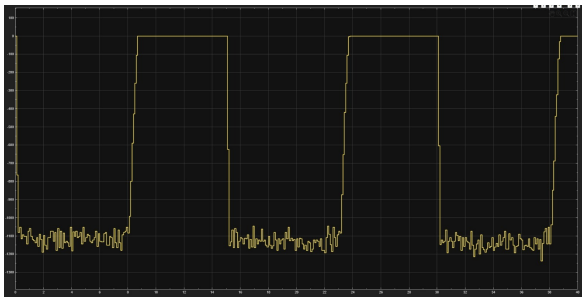
(b) Reverse direction configuration

Figure 6: Simulink configurations verifying motor direction control

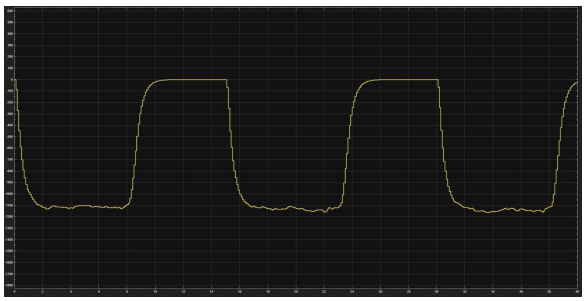
5 Speed Measurement and Filtering

The motor speed was measured using the incremental encoder attached to the motor shaft. The encoder generates pulse signals proportional to the angular velocity. These pulses were counted using interrupt-driven routines on the Arduino, and the speed was calculated in revolutions per minute (RPM).

Due to the discrete nature of encoder pulses and potential electrical noise, the raw speed signal contains significant fluctuations. To improve signal quality and reduce noise, a first-order low-pass filter was applied to the measured speed. Two scopes were used to observe the system behavior: one displaying the raw (unfiltered) speed and the other showing the filtered speed.



(a) Unfiltered speed measurement



(b) Filtered speed measurement

Figure 7: Measured motor speed before and after low-pass filtering

The comparison clearly demonstrates the effectiveness of the low-pass filter in attenuating high-frequency noise while preserving the fundamental speed information.

6 Effect of Filter Time Constant

The effect of changing the filter time constant on the measured motor speed was investigated to study the trade-off between noise attenuation and system response speed. Four different time constants were tested to evaluate their impact on filtering performance.

6.1 First-Order Low-Pass Filter Design

The general continuous-time first-order low-pass filter is given by:

$$G(s) = \frac{1}{\tau s + 1}$$

Using Zero-Order Hold (ZOH) discretization with a sampling time of $T = 0.1$ s, the corresponding discrete-time transfer function is:

$$G(z) = \frac{1 - e^{-T/\tau}}{z - e^{-T/\tau}}$$

The following four cases were analyzed:

6.2 Case 1: $\tau = 0.2$ s

$$G_1(s) = \frac{1}{0.2s + 1}$$

$$G_1(z) = \frac{1 - e^{-0.1/0.2}}{z - e^{-0.1/0.2}} = \frac{0.3935}{z - 0.6065}$$

6.3 Case 2: $\tau = 0.5$ s

$$G_2(s) = \frac{1}{0.5s + 1}$$

$$G_2(z) = \frac{1 - e^{-0.1/0.5}}{z - e^{-0.1/0.5}} = \frac{0.1813}{z - 0.8187}$$

6.4 Case 3: $\tau = 1$ s

$$G_3(s) = \frac{1}{s + 1}$$

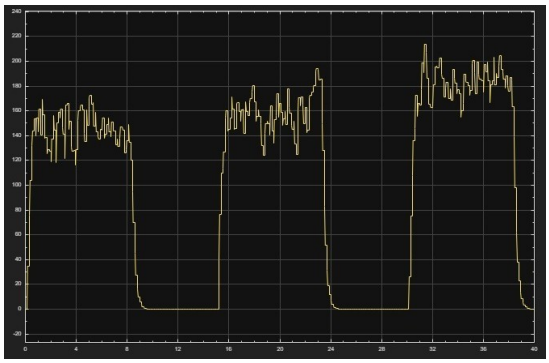
$$G_3(z) = \frac{1 - e^{-0.1}}{z - e^{-0.1}} = \frac{0.0952}{z - 0.9048}$$

6.5 Case 4: $\tau = 2$ s

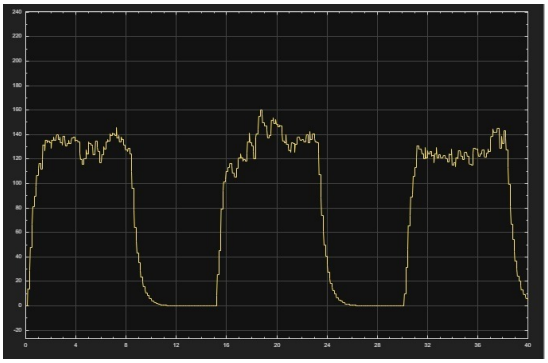
$$G_4(s) = \frac{1}{2s + 1}$$

$$G_4(z) = \frac{1 - e^{-0.1/2}}{z - e^{-0.1/2}} = \frac{0.0488}{z - 0.9512}$$

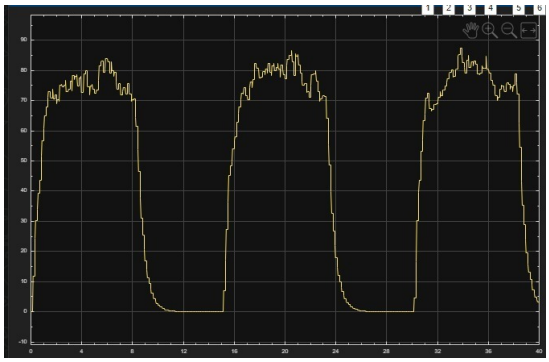
6.6 Forward Direction Response



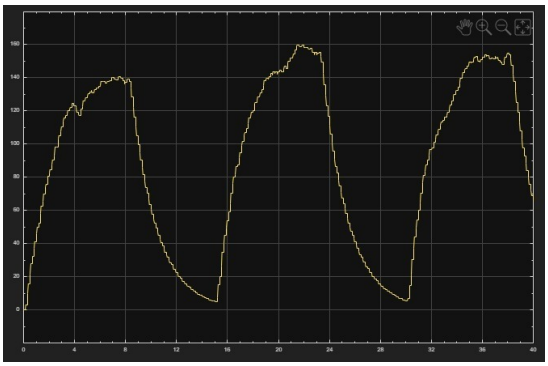
(a) $\tau = 0.2$ s



(b) $\tau = 0.5$ s



(c) $\tau = 1$ s

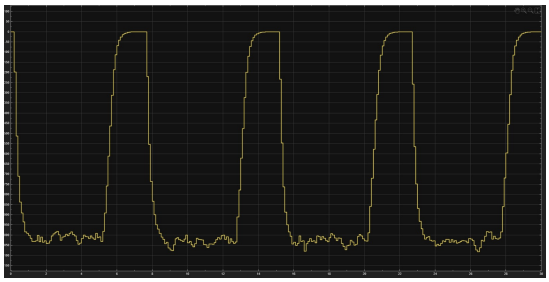


(d) $\tau = 2$ s

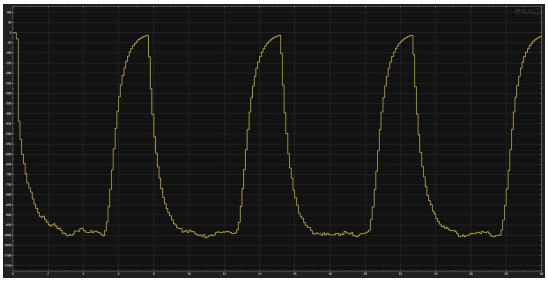
Figure 8: Forward-direction motor speed response for different filter time constants

As the time constant τ increases, the discrete-time pole $e^{-T/\tau}$ moves closer to unity, resulting in smoother speed signals but a slower dynamic response. A smaller time constant provides faster tracking but with more residual noise, while a larger time constant yields better noise suppression at the expense of increased response delay.

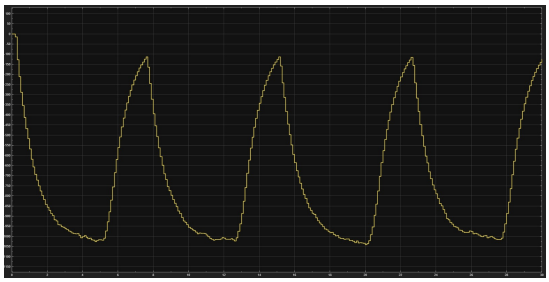
6.7 Reverse Direction Response



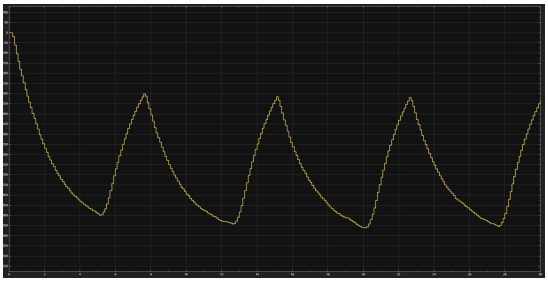
(a) $\tau = 0.2$ s



(b) $\tau = 0.5$ s



(c) $\tau = 1$ s



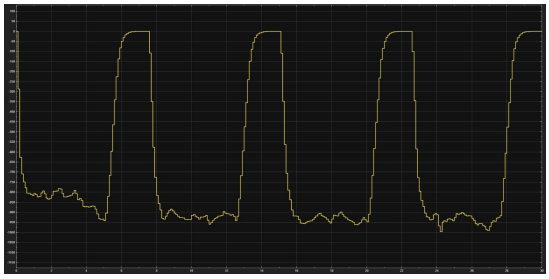
(d) $\tau = 2$ s

Figure 9: Reverse-direction motor speed response for different filter time constants

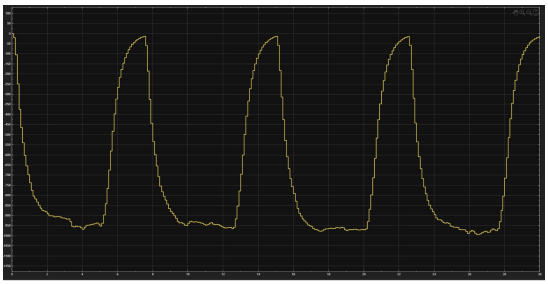
The results for reverse direction operation are consistent with those observed in forward direction. Increasing the filter time constant improves noise suppression in both directions, at the expense of increased response delay. The choice of time constant should balance noise rejection requirements with the need for responsive feedback.

6.8 Verification Using MATLAB

To verify the correctness of the hand-calculated Z-transform results, the same continuous-time filters were discretized in MATLAB using the `c2d` command with ZOH discretization. The comparison was performed for the first two time constants.



(a) $\tau = 0.2$ s



(b) $\tau = 0.5$ s

Figure 10: Discrete-time filter responses obtained using MATLAB `c2d` with ZOH discretization

The MATLAB results closely match the analytically derived discrete-time models, confirming the validity of the hand calculations and the correct implementation of the discretization procedure.

7 System Identification

System identification is the process of developing mathematical models of dynamic systems based on measured input-output data. In this project, the MATLAB System Identification Toolbox was used to estimate the transfer function of the DC motor from experimental data collected during open-loop operation.

Different transfer function structures with various pole–zero configurations were tested to determine which model best represents the system dynamics and steady-state behavior. Each case represents a distinct system structure, and the goodness of fit was evaluated by comparing the model output with the measured data.

7.1 Tested Transfer Function Structures

7.1.1 Case 1: Second-Order System with One Zero

The transfer function is given by:

$$G_1(s) = \frac{-3.341s + 98.61}{s^2 + 12s + 22.53}$$

This model includes a zero in the numerator, which can capture inverse response or lead-lag dynamics if present in the system.

7.1.2 Case 2: Second-Order System without Zeros

The transfer function is expressed as:

$$G_2(s) = \frac{69.92}{s^2 + 8.95s + 15.98}$$

This is a standard second-order model without zeros, suitable for systems exhibiting underdamped or overdamped behavior.

7.1.3 Case 3: Third-Order System with Two Zeros

The transfer function is defined as:

$$G_3(s) = \frac{-2.354s^2 + 81.16s + 30.21}{s^3 + 10.26s^2 + 22.83s + 6.867}$$

This higher-order model can capture more complex dynamics but may also introduce additional complexity and potential overfitting.

7.1.4 Case 4: First-Order System

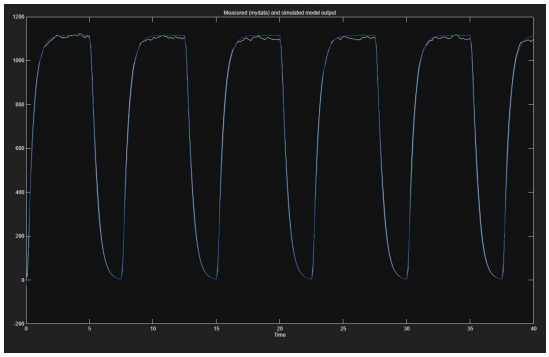
The transfer function is written as:

$$G_4(s) = \frac{7.594}{s + 1.712}$$

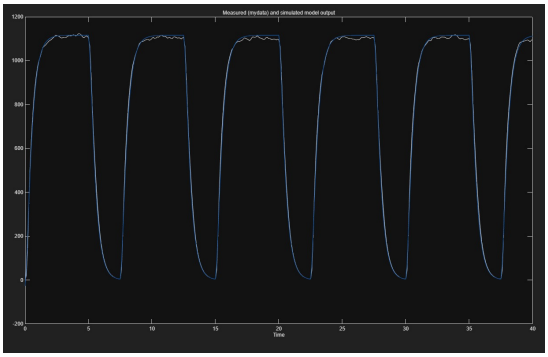
This is the simplest model structure, representing a first-order lag. It is often sufficient for DC motor speed control applications.

7.2 Pulse Response Comparison

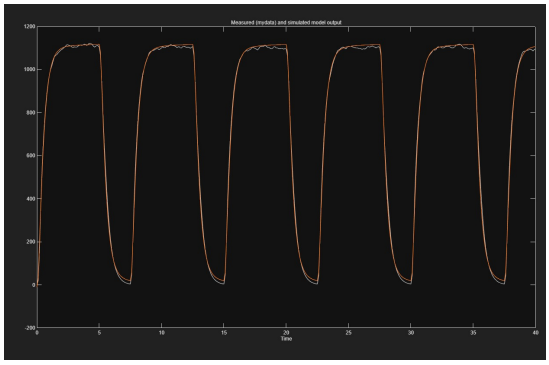
To evaluate the accuracy of each model, a rectangular pulse input was applied to the motor, and the measured speed response was compared with the predicted response from each identified model.



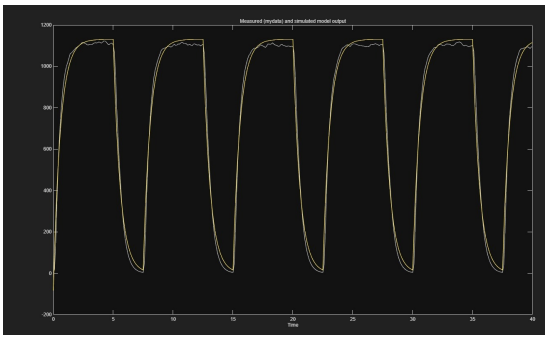
(a) Case 1: Second-order with one zero



(b) Case 2: Second-order without zeros



(c) Case 3: Third-order with two zeros



(d) Case 4: First-order

Figure 11: Comparison of pulse responses for the four transfer function cases

7.3 Model Selection

Based on the comparison of pulse responses, the first-order model (Case 4) was selected for controller design. Although higher-order models may provide slightly better fit to the data, the first-order model offers sufficient accuracy for control purposes while maintaining simplicity. The selected model is:

$$G_p(s) = \frac{K}{\tau s + 1} = \frac{4.436}{s + 0.584}$$

This form relates the motor speed (in RPM) to the PWM input and was used as the basis for PI controller design.

8 PI Controller Design

Based on the identified first-order model of the DC motor, a PI controller was designed to achieve the desired closed-loop performance specifications. The design procedure involves determining the controller gains that provide adequate damping, fast settling time, and zero steady-state error.

8.1 System Model and Controller Structure

The plant is modeled as a first-order transfer function:

$$G_p(s) = \frac{K}{\tau s + 1} \tag{1}$$

A PI controller is chosen in the form:

$$G_c(s) = K_p + \frac{K_i}{s} \tag{2}$$

The open-loop transfer function is:

$$G_{OL}(s) = G_c(s)G_p(s) = \frac{K(K_p s + K_i)}{s(\tau s + 1)} \tag{3}$$

8.2 Closed-Loop Characteristic Equation

The closed-loop transfer function is:

$$G_{CL}(s) = \frac{G_{OL}(s)}{1 + G_{OL}(s)} \tag{4}$$

The characteristic equation is obtained as:

$$1 + G_{OL}(s) = 0 \quad \Rightarrow \quad \tau s^2 + (1 + KK_p)s + KK_i = 0 \tag{5}$$

Dividing by τ yields:

$$s^2 + \frac{1 + KK_p}{\tau}s + \frac{KK_i}{\tau} = 0 \tag{6}$$

8.3 Design Based on Settling Time Specification

The design specification requires that the closed-loop settling time be 80% of the open-loop settling time. From the open-loop step response of the identified first-order system, the open-loop settling time was measured. Let this settling time be denoted as $T_{s,OL}$.

The desired closed-loop settling time is:

$$T_{s,CL} = 0.8 \times T_{s,OL} \tag{7}$$

For the open-loop first-order system with time constant $\tau = 0.85$ s, the settling time (2% criterion) is:

$$T_{s,OL} = 4\tau = 4 \times 0.584 = 2.336 \text{ s} \tag{8}$$

Therefore, the desired closed-loop settling time is:

$$T_{s,CL} = 0.8 \times 2.336 = 1.869 \text{ s} \tag{9}$$

For a second-order closed-loop system, the settling time is approximately:

$$T_{s,CL} = \frac{4}{\zeta\omega_n} \tag{10}$$

Substituting the desired settling time:

$$1.869 = \frac{4}{\zeta\omega_n} \tag{11}$$

Solving for the product $\zeta\omega_n$:

$$\zeta\omega_n = \frac{4}{1.869} = 2.14 \tag{12}$$

The standard second-order system characteristic equation is:

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0 \tag{13}$$

By comparing coefficients with Equation 6:

$$2\zeta\omega_n = \frac{1 + KK_p}{\tau} \tag{14}$$

$$\omega_n^2 = \frac{KK_i}{\tau} \tag{15}$$

Solving for the PI controller gains:

$$K_p = \frac{2\zeta\omega_n\tau - 1}{K} \tag{16}$$

$$K_i = \frac{\omega_n^2\tau}{K} \tag{17}$$

From Equation 12, we have $\zeta\omega_n = 2.14$. To complete the design, we need to specify the damping ratio ζ (which determines the overshoot), and then calculate ω_n from:

$$\omega_n = \frac{2.14}{\zeta} \tag{18}$$

Two different designs are developed based on different overshoot specifications, both satisfying the same settling time requirement.

8.4 Controller Design for Different Overshoot Specifications

Two controller designs were developed corresponding to different maximum overshoot specifications, both satisfying the same settling time requirement of $T_{s,CL} = 2.72$ s (i.e., $\zeta\omega_n = 1.47$). The maximum percent overshoot for a second-order system is related to the damping ratio by:

$$\text{OS}_{\text{max}} = 100 \times e^{-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}}$$

8.4.1 Case 1: Maximum Overshoot = 5%

For a maximum overshoot of 5%, the corresponding damping ratio is:

$$\zeta = 0.69 \tag{19}$$

Selecting a natural frequency $\omega_n = 5.5$ rad/s to achieve a settling time of approximately 1 s, the PI controller gains are calculated using Equations 16 and 17:

$$K_p = 0.3382 \tag{20}$$

$$K_i = 1.265 \tag{21}$$

The resulting controller is:

$$G_c(s) = 0.3382 + \frac{1.265}{s} \tag{22}$$

Discussion: The closed-loop response exhibits low overshoot (approximately 5%) and smooth transient behavior. The higher damping ratio leads to reduced oscillations at the expense of a slightly slower rise time compared to less damped systems. This controller provides a conservative, stable response suitable for applications where overshoot must be minimized.

8.4.2 Case 2: Maximum Overshoot = 10%

For a maximum overshoot of 10%, the damping ratio becomes:

$$\zeta = 0.59 \tag{23}$$

Using the same natural frequency $\omega_n = 5.5$ rad/s, the corresponding PI controller gains are:

$$K_p = 0.3382 \tag{24}$$

$$K_i = 1.7323 \tag{25}$$

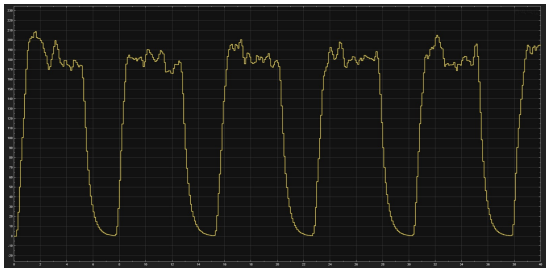
The resulting controller is:

$$G_c(s) = 0.3382 + \frac{1.7323}{s} \tag{26}$$

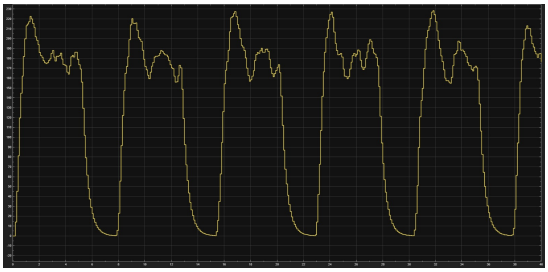
Discussion: Reducing the damping ratio increases the system aggressiveness, resulting in faster rise time and higher overshoot (approximately 10%). This design provides quicker response but introduces more oscillatory behavior compared to the 5% overshoot case. The choice between the two designs depends on the specific application requirements regarding speed of response versus overshoot tolerance.

8.5 Experimental Validation

The discrete-time controllers were implemented in Simulink and deployed to the Arduino hardware. The closed-loop step responses were measured experimentally and compared with simulation results.



(a) 5% overshoot design



(b) 10% overshoot design

Figure 12: Experimental closed-loop step responses for the two PI controller designs

The experimental results confirm close agreement between the theoretical design and actual system performance, validating the correctness of the controller design procedure and the discretization method.

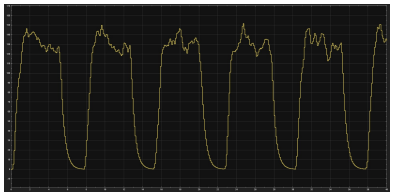
9 Effect of Different Speed Setpoints

In this section, the performance of the PI-controlled DC motor is evaluated under different reference speed setpoints. The motor is driven by a PWM signal with a maximum input voltage of 12 V, which corresponds to the maximum achievable motor speed.

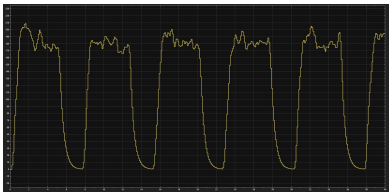
Three reference levels were tested to assess tracking performance and to identify potential limitations due to actuator constraints:

- 50% of maximum speed
- 70% of maximum speed
- 100% of maximum speed

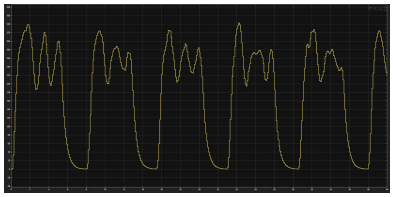
9.1 Setpoint Tracking Performance



(a) 50% speed setpoint



(b) 70% speed setpoint



(c) 100% speed setpoint

Figure 13: Closed-loop motor speed response for different reference setpoints

9.2 Analysis and Discussion

For the 50% and 70% setpoints, the controller achieves accurate tracking with minimal steady-state error and smooth transient behavior. The system response closely follows the desired trajectory, and the PI controller successfully eliminates steady-state error through integral action.

At the 100% setpoint, however, the control signal reaches the actuator limit corresponding to the maximum available input voltage (12 V). This results in **actuator saturation**, which limits the achievable control effort and causes the following effects:

- Slower rise time compared to lower setpoints
- Deviation from the ideal reference trajectory
- Potential integrator windup if not properly handled

The observed saturation behavior highlights the practical limitations of the motor drive and emphasizes the importance of considering actuator constraints in controller design. In applications where full-range operation is required, anti-windup techniques or model predictive control may be necessary to handle saturation effects more effectively.

10 Conclusion

This project successfully demonstrated the design and implementation of a proportional–integral (PI) controller for DC motor speed control. The project encompassed several key stages, including hardware setup, system identification, controller design, and performance evaluation.

System identification techniques were employed to obtain an accurate first-order mathematical model of the motor dynamics from experimental data. Multiple transfer function structures were evaluated, and the first-order model was selected based on its simplicity and adequate representation of the system behavior.

The PI controller parameters were analytically derived by matching the closed-loop characteristic equation to a standard second-order system with specified damping ratio and natural frequency. Two controller designs were developed corresponding to maximum overshoot specifications of 5% and 10%. The results demonstrated that a lower overshoot specification leads to a more damped response with reduced oscillations, while a higher overshoot specification provides faster rise time at the expense of increased transient overshoot.

Discrete-time implementation was carried out using Zero-Order Hold discretization with a sampling time of 0.1 s. The hand-calculated discrete-time models were validated through MATLAB simulations using the `c2d` function, and close agreement was observed, confirming the correctness of the design procedure.

The effect of low-pass filter time constants on noise attenuation was investigated. It was found that increasing the time constant improves noise suppression but introduces response delay. A balanced choice of filter time constant is necessary to achieve both adequate noise rejection and responsive feedback.

The controller performance was evaluated under different speed setpoints corresponding to 50%, 70%, and 100% of the maximum motor input voltage. While the controller achieved accurate tracking for lower setpoints, saturation effects were observed at full-scale input due to actuator limitations. This highlights the practical constraints of real-world motor control systems and emphasizes the importance of considering saturation and nonlinear effects during controller design.

Overall, the designed PI controller demonstrated effective speed regulation, good robustness, and satisfactory transient performance. The project successfully integrated theoretical control design with practical implementation using Arduino and Simulink.

References

1. University of Michigan, *DC Motor Control Laboratory – CTMS*. Available at: https://ctms.engin.umich.edu/CTMS/index.php?aux=Activities_DCmotorB

2. DroneBot Workshop, *DC Motors and H-Bridge Motor Drivers*. Available at: https://dronebotworkshop.com/dc-motors-l298n-h-bridge/

3. Shadab N. Sayyad and W. Gavhane, *DC Motor Speed Control Using PI Controller*, International Journal of Innovations in Engineering Research and Technology (IJIERT), 2022. Available at: https://media.neliti.com/media/publications/429578-dc-motor-speed-control-using-pi-controll-b46d8ae0.pdf

4. MATLAB Documentation, *Continuous to Discrete Conversion Using c2d*. Available at: https://www.mathworks.com/help/control/ref/c2d.html

5. MATLAB Documentation, *System Identification Toolbox*. Available at: https://www.mathworks.com/help/ident/